In [ ]:

```
# Initialize Otter
import otter
grader = otter.Notebook("lab05.ipynb")
```

Fundamentos en ciencias de datos

Laboratorio 5: Simulaciones

¡Bienvenido al Laboratorio 5!

Revisaremos la iteración (https://www.inferentialthinking.com/chapters/09/2/Iteration.html) y las simulaciones (https://www.inferentialthinking.com/chapters/09/3/Simulation.html), así como introduciremos el concepto de aleatoriedad (https://www.inferentialthinking.com/chapters/09/Randomness.html).

In [1]:

```
# These lines import the Numpy and Datascience modules.
import numpy as np
from datascience import *

# These lines do some fancy plotting magic
import matplotlib
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
```

Ingresa tu nombre con apellido y número de carnet en las variables dadas en la celda siguiente: Ejemplo:

```
NombreApellidos="Ada Lovelace"
NumeroCarnet="3141592"
Email="Ada.Lovelace@udea.edu.co"
```

In [2]:

```
# BEGIN SOLUTION NO PROMPT
Nombre = "Ada Lovelace"
Carnet =  "3141592.... "
# END SOLUTION
""" # BEGIN PROMPT
Nombre = "..."
Carnet = "..."
""" # END PROMPT
```

Out[2]:

```
' # BEGIN PROMPT\nNombre = "..."\nCarnet = "..."\n'
```

1. Nachos y Condicionales

En esta sección, nos centraremos en el concepto de condicionales, que es esencialmente cómo tomamos decisiones en los programas utilizando estructuras como "if...else". Imagínate que tienes un plato de nachos y estás decidiendo si agregar más salsa o queso. Usarías un condicional para "decidir" qué acción tomar dependiendo de si hay suficiente salsa o queso. Este tipo de lógica es lo que intentaremos aplicar en nuestros ejercicios.

En Python, el booleano es un tipo de dato con solo dos valores posibles: `True` y `False`. Las expresiones que contienen operadores de comparación como `<` (menor que), `>` (mayor que) y `==` (igual a) se evalúan en valores booleanos. ¡A continuación puedes encontrar una lista de operadores de comparación comunes!

Comparación	Operador	Ejemplo verdadero	Ejemplo falso
Menor que	`<`	`2 < 3`	`2 < 2`
Mayor que	`>`	`3 > 2`	`3 > 3`
Menor o igual que	`<=`	`2 <= 2`	`3 <= 2`
Mayor o igual que	`>=`	`3 >= 3`	`2 >= 3`
Igual	`==`	`3 == 3`	`3 == 2`
No igual	`!=`	`3 != 2`	`2 != 2`

Ejecuta la celda a continuación para ver un ejemplo de un operador de comparación en acción.

In [3]:

```
3 > (1 + 1)
```

Out[3]:

```
True
```

Podemos incluso asignar el resultado de una operación de comparación a una variable. Nota que `==` y `=` **no** son lo mismo.

In [4]:

```
result = 10 / 2 == 5
result
```

Out[4]:

```
True
```

Así como los operadores aritméticos pueden aplicarse a cada elemento de un arreglo, los operadores de comparación también pueden utilizarse en arreglos para comparar todo un arreglo con algún valor. El resultado de esta comparación es un arreglo de valores booleanos.

In [5]:

```
make_array(1, 5, 7, 8, 3, -1) > 3
```

Out[5]:

```
array([False,  True,  True,  True, False, False], dtype=bool)
```

Spongamos que ves un plato de nachos esperando en la mesa del comedor. Supongamos que cada vez que tomas un nacho del plato, puede tener solo **queso**, solo **guacamole**, **ambos** queso y guacamole, o **ninguno** de los dos.

Intentemos simular sacar nachos del cuenco al azar usando la función `np.random.choice(...)`.

`np.random.choice`

`np.random.choice` elige un elemento al azar del arreglo dado. Es igualmente probable elegir cualquiera de los elementos. Ejecuta la celda a continuación varias veces y observa cómo cambian los resultados. *Consejo:* Para ejecutar una celda varias veces, puedes usar el atajo de teclado `ctrl + return`.

In [6]:

```
nachos = make_array('queso', 'gucamole', 'ambos', 'ninguno')
np.random.choice(nachos)
```

Out[6]:

```
'queso'
```

Para repetir este proceso múltiples veces, pasa un entero `n` como segundo argumento para devolver `n` elecciones aleatorias diferentes. Por defecto, `np.random.choice` realiza muestras **con reemplazo** y devuelve un *array* de elementos. Tomar muestras **con reemplazo** significa que, después de elegir un elemento, este se devuelve al lugar de donde se está muestreando y puede ser elegido nuevamente en el futuro. Si tomamos `n` muestras con reemplazo, cada vez, cada elemento tiene la misma probabilidad de ser seleccionado.

Ejecuta la siguiente celda para ver un ejemplo de muestreo con reemplazo 10 veces del array `nachos`.

In [7]:

```
np.random.choice(nachos, 10)
```

Out[7]:

```
array(['ninguno', 'ninguno', 'gucamole', 'queso', 'queso', 'gucamole',
       'gucamole', 'queso', 'ambos', 'ninguno'],
      dtype='<U8')
```

Para contar la cantidad de veces que un cierto tipo de nacho es elegido aleatoriamente, podemos usar `np.count_nonzero`.

`np.count_nonzero`

`np.count_nonzero` cuenta el número de valores diferentes de cero que aparecen en un arreglo. Cuando se pasa un arreglo de valores booleanos a través de la función, contará el número de valores `True` (recuerda que en Python, **`True` se codifica como 1 y `False` se codifica como 0.**)

Ejecuta la siguiente celda para ver un ejemplo que utiliza `np.count_nonzero`.

In [8]:

```
np.count_nonzero(make_array(True, False, False, True, True))
```

Out[8]:

```
3
```

Pregunta 1.1 Supongamos que seleccionamos diez nachos al azar y almacenamos los resultados en un arreglo llamado `diez_nachos`.

Usando código, encuentra cuántos de esos nachos tienen solo queso. Almacena el resultado en una variable llamada `numero_quesos` No ingreses la respuesta manualmente.

Pista: Utiliza un operador de comparación (como `==`, `<`, etc.) junto con el método `np.count_nonzero`.

In [74]:

```
# BEGIN SOLUTION NO PROMPT


# Supongamos que la tabla 'nachos' tiene una columna llamada 'Type' con valores 'cheese' y otros.
# Aquí es donde se seleccionarían diez nachos al azar (esto es solo un ejemplo, en un entorno real
# la tabla y el método para seleccionar el subconjunto tendrían que definirse).

diez_nachos = np.random.choice(nachos, 10)

# Calcula el número de nachos solo con queso.
numero_quesos = np.count_nonzero(diez_nachos == 'queso')

# END SOLUTION
""" # BEGIN PROMPT
diez_nachos = np.random.choice(nachos, 10)
numero_quesos=...
""" # END PROMPT
```

Out[74]:

```
' # BEGIN PROMPT\ndiez_nachos = np.random.choice(nachos, 10)\nnumero_quesos=...\n'
```

In [ ]:

```
grader.check("q0")
```

Instrucciones Condicionales

Una instrucción condicional es una declaración de varias líneas que permite a Python elegir entre diferentes alternativas basadas en el valor de verdad de una expresión.

Aquí tienes un ejemplo básico.

```
def sign(x):
    if x > 0:
        return 'Positive'
    else:
        return 'Negative'
```

Si la entrada `x` es mayor que `0`, devolvemos la cadena `'Positive'`. De lo contrario, devolvemos `'Negative'`.

Si queremos probar múltiples condiciones a la vez, utilizamos el siguiente formato general.

```
if <if expression>:
    <if body>
elif <elif expression 0>:
    <elif body 0>
elif <elif expression 1>:
    <elif body 1>
...
else:
    <else body>
```

Solo se evaluará el cuerpo de la primera expresión condicional que sea verdadera. Cada expresión `if` y `elif` se evalúa y considera en orden, comenzando desde la parte superior. `elif` solo se puede usar si le precede una cláusula `if`. Tan pronto como se encuentre un valor verdadero, se ejecuta el cuerpo correspondiente, y el resto de la instrucción condicional se omite. Si ninguna de las expresiones `if` o `elif` es verdadera, entonces se ejecuta el `else body`.

Para más ejemplos y explicaciones, consulta la sección sobre instrucciones condicionales aquí (https://inferentialthinking.com/chapters/09/1/Conditional_Statements.html).

Pregunta 1.2 Completa la siguiente declaración condicional. Si la cantidad de nachos con queso en `diez_nachos` es menor que `5`, asigne a la variable `mas` la cadena `'mas por favor'`. Usa la declaración if para hacerlo.

Pista: Deberías estar utilizando `numero_quesos` de la Pregunta 1.

In [79]:

```
# BEGIN SOLUTION NO PROMPT

if numero_quesos < 5:
    mas = 'mas por favor'

# END SOLUTION
""" # BEGIN PROMPT

if ...:
    mas = 'mas por favor'
mas
""" # END PROMPT
```

Out[79]:

```
" # BEGIN PROMPT\n\nif ...:\n    mas = 'mas por favor'\nmas\n"
```

In [ ]:

```
grader.check("q1")
```

Pregunta 1.3 Escribe una función llamada `nacho_reaccion` que devuelva una reacción (como una cadena de texto) basada en el tipo de nacho que se pase como argumento. Usa la tabla a continuación para hacer coincidir el tipo de nacho con la reacción correspondiente.

Tipo de Nacho	Reacción
queso	¡quesudo!
gucamole	¡cremosito!
ambos	¡wow!
ninguno	¡meh!

Pista: Si no pasa la prueba, verifica la ortografía de tus reacciones.

In [ ]:

```

```

In [107]:

```
# BEGIN SOLUTION NO PROMPT

# def nacho_reaccion(nacho_type):
#     reactions = {
#         'queso': '¡quesudo!',
#         'guacamole': '¡cremosito!',
#         'ambos': '¡wow!',
#         'ninguno': 'meh.'
#     }
#     return reactions.get(nacho_type, 'tipo desconocido')
def nacho_reaccion(nacho):
    if nacho == "queso":
        return "¡quesudo!"
    elif nacho=="guacamole":
        return "¡cremosito!"
    elif nacho=="ambos":
        return "¡wow!"
    elif nacho=="ninguno":
        return "¡meh!"
    else:
        return "No esta la reaccion"
# END SOLUTION
""" # BEGIN PROMPT
def nacho_reaccion(nacho):
    if nacho == "queso":
        return ...
    elif ...

nacho_quesudo = nacho_reaccion('guacamole')
nacho_quesudo
""" # END PROMPT
```

Out[107]:

```
' # BEGIN PROMPT\ndef nacho_reaccion(nacho):\n    if nacho == "queso":\n        return ...\n    elif .
.\n    \nnacho_quesudo = nacho_reaccion(\'guacamole\')\nnacho_quesudo\n'
```

In [ ]:

```
grader.check("q2")
```

Pregunta 1.4 Crea una tabla `diez_nachos_reacciones` que consista en los nachos de `diez_nachos` así como las reacciones para cada uno de esos nachos. Las columnas deben llamarse `Nachos` y `Reacciones`.

Pista: Considera usar el método `apply`, que retorna un arreglo.

In [104]:

```
# BEGIN SOLUTION NO PROMPT
diez_nachos_reacciones = Table().with_column('Nachos', diez_nachos)
reacciones = diez_nachos_reacciones.apply(nacho_reaccion, 'Nachos')
diez_nachos_reacciones = diez_nachos_reacciones.with_column('Reacciones', reacciones)

diez_nachos_reacciones
# END SOLUTION
""" # BEGIN PROMPT
diez_nachos_tbl = Table().with_column('Nachos', diez_nachos)
diez_nachos_reacciones = ...
diez_nachos_reacciones

""" # END PROMPT
```

Out[104]:

```
" # BEGIN PROMPT\ndiez_nachos_tbl = Table().with_column('Nachos', diez_nachos)\ndiez_nachos_reacciones
= ...\ndiez_nachos_reacciones\n\n"
```

In [ ]:

```
grader.check("q14")
```

Pregunta 1.5 Utilizando código, encuentra el número de reacciones 'wow!' para los nachos en `diez_nachos_reactions` .

In [106]:

```
# BEGIN SOLUTION NO PROMPT

numero_wow_reacciones = diez_nachos_reacciones.where('Reacciones', 'Wow!').num_rows
numero_wow_reacciones

# END SOLUTION
""" # BEGIN PROMPT
numero_wow_reacciones = ...
numero_wow_reacciones
""" # END PROMPT
```

Out[106]:

```
' # BEGIN PROMPT\nnumero_wow_reacciones = ...\nnumero_wow_reacciones\n'
```

In [ ]:

```
grader.check("q15")
```

2. Simulaciones y Ciclos For

Usando una declaración `for` , podemos realizar una tarea varias veces. Esto se conoce como iteración. La estructura general de un ciclo for es:

```
for <placeholder> in <array>:
```

seguido por líneas de código indentadas que se repiten para cada elemento del `array` que se está iterando. Puedes leer más sobre ciclos for aquí (https://www.inferentialthinking.com/chapters/09/2/Iteration.html).

NOTA: A menudo usamos `i` como el `placeholder` (Marcador de posición) en nuestros ejemplos de clase, ¡pero podrías nombrarlo de cualquier forma! Algunos ejemplos se pueden encontrar a continuación.

Un uso de la iteración es recorrer un conjunto de valores. Por ejemplo, podemos imprimir todos los colores del arcoíris.

In [108]:

```
arcoiris = make_array("rojo", "naranja", "amarillo", "verde", "azul", "índigo", "violeta")

for color in arcoiris:
    print(color)
```

```
rojo
naranja
amarillo
verde
azul
índigo
violeta
```

Podemos ver que la parte indentada del ciclo `for` , conocida como el cuerpo, se ejecuta una vez por cada elemento en `arcoiris` . El nombre `color` se le asigna al siguiente valor en `arcoiris` al inicio de cada iteración. Nota que el nombre `color` es arbitrario; podríamos fácilmente haberlo nombrado de otra manera. Sea cual sea el nombre que elijamos, necesitamos usarlo de manera consistente a lo largo del ciclo `for` .

In [109]:

```
for otro_nombre in arcoiris:
    print(otro_nombre)
```

```
rojo
naranja
amarillo
verde
azul
índigo
violeta
```

En general, sin embargo, nos gustaría que el nombre de la variable sea algo informativo.

Pregunta 2.1

En la siguiente celda, hemos cargado el texto de *las mil y una noche*, lo hemos separado en palabras individuales y hemos almacenado estas palabras en un arreglo llamado `palabras`. Usando un ciclo for, asigna a `mas_de_cinco` el número de palabras en la novela que tienen más de 5 letras.

Sugerencia: Puedes encontrar el número de letras en una palabra con la función len.

Sugerencia: ¿Cómo puedes utilizar mas_de_cinco para llevar un registro del número de palabras que tienen más de cinco letras?

In [111]:

```
# BEGIN SOLUTION NO PROMPT
import urllib.request
# Leer el contenido directamente desde la URL del Proyecto Gutenberg
url = "https://www.gutenberg.org/cache/epub/47287/pg47287.txt"
respuesta = urllib.request.urlopen(url)
texto = respuesta.read().decode("utf-8")

# Separar en palabras individuales
palabras = np.array(texto.split())

# Contar cuántas tienen más de 5 letras
mas_de_cinco = 0

for palabra in palabras:
    if len(palabra) > 5:
        mas_de_cinco += 1

mas_de_cinco

# END SOLUTION
""" # BEGIN PROMPT
import urllib.request
url = "https://www.gutenberg.org/cache/epub/47287/pg47287.txt"
respuesta = urllib.request.urlopen(url)
texto = respuesta.read().decode("utf-8")

# Separar en palabras individuales
palabras = np.array(texto.split())
""" # END PROMPT
```

Out[111]:

```
' # BEGIN PROMPT\nimport urllib.request\nurl =
"https://www.gutenberg.org/cache/epub/47287/pg47287.txt"\nrespuesta =
urllib.request.urlopen(url)\ntexto = respuesta.read().decode("utf-8")\n\n# Separar en palabras individ
uales\npalabras = np.array(texto.split())\n'
```

In [ ]:

```
grader.check("q5")
```

Otra forma en la que podemos usar los bucles `for` es repitiendo líneas de código muchas veces. Recordemos la estructura de un bucle `for`:

`for <placeholder> in <array>:` seguido por líneas de código indentadas que se repiten para cada elemento del arreglo sobre el que se está iterando.

A veces, no nos interesa cuál es el valor del marcador de posición. En cambio, aprovechamos el hecho de que el bucle `for` se repetirá tantas veces como la longitud de nuestro arreglo. En la siguiente celda, iteramos a través de un arreglo de longitud 5 e imprimimos "Hello, world!" en cada iteración, pero no necesitamos usar el marcador de posición `i` en el cuerpo de nuestro bucle `for`.

In [113]:

```
for i in np.arange(5):
    print("Hola mundo !")
```

```
Hola mundo !
Hola mundo !
Hola mundo !
Hola mundo !
Hola mundo !
```

Pregunta 2.2

Utilizando una simulación con 10,000 experimentos, asigna a la variable `num_diferentes` el número de veces que dos palabras, seleccionadas al azar con reemplazo del texto "Las mil y una noches", no son iguales.

Pista 1: ¿Qué función usamos en la sección 1 para muestrear al azar con reemplazo de un arreglo?

Pista 2: Recuerda que `!=` verifica la desigualdad entre dos elementos.

In [114]:

```
# BEGIN SOLUTION NO PROMPT


url = "https://www.gutenberg.org/cache/epub/47287/pg47287.txt"
respuesta = urllib.request.urlopen(url)
texto = respuesta.read().decode("utf-8")

# Separar en palabras individuales
palabras = np.array(texto.split())

numero_repeticiones = 10_000
numero_diferentes = 0

# Simulación: comparar dos palabras aleatorias en cada repetición
for _ in range(numero_repeticiones):
    palabra_1 = np.random.choice(palabras)
    palabra_2 = np.random.choice(palabras)
    if palabra_1 != palabra_2:
        numero_diferentes += 1

# Mostrar resultado
numero_diferentes

# END SOLUTION
""" # BEGIN PROMPT
numero_repeticiones = 10_000
numero_diferentes = 0

for ... in ...:
    ...
numero_diferentes
""" # END PROMPT
```

Out[114]:

```
' # BEGIN PROMPT\nnumero_repeticiones = 10_000\nnumero_diferentes = 0\n\nfor ... in ...:\n    ...\nnum
ero_diferentes\n'
```

In [ ]:

```
grader.check("q6")
```

Muestreo en datos biológicos

En esta sección introduciremos el concepto de muestreo, una herramienta fundamental en estudios biológicos. En la práctica, rara vez podemos medir toda una población de organismos (plantas, animales, células, etc.). Por ello, tomamos muestras y usamos sus propiedades para estimar parámetros poblacionales. Trabajaremos con un conjunto de datos clásico en biología: el dataset Iris. Este contiene mediciones morfológicas de tres especies de flores del género Iris:

- Iris setosa
- Iris versicolor
- Iris virginica

Cada flor tiene medidas de:

- Longitud y ancho del sépalo
- Longitud y ancho del pétalo

Si deseas repasar los distintos tipos de muestreo (aleatorio, estratificado, sistemático), puedes consultar la Sección 10 del libro de texto.

In [117]:

```
datos_iris = Table.read_table("https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris
.csv")

# Mostrar las primeras 3 filas
print("Datos de flores Iris:")
datos_iris.show(3)
```

```
Datos de flores Iris:
```

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa

... (147 rows omitted)

En lugar de contar con información completa de cada flor del conjunto de datos Iris, imaginemos que solo tenemos acceso a un subconjunto más pequeño de muestras. Este conjunto contiene datos morfológicos de 150 flores, y aunque no es una cantidad abrumadora, en estudios reales de campo o laboratorio no siempre es posible medir todos los ejemplares de una población biológica. A menudo debemos trabajar solo con una muestra parcial.

Si queremos hacer estimaciones sobre alguna propiedad numérica de la población —como la longitud promedio del pétalo o del sépalo — es probable que tengamos que basarnos únicamente en esta muestra.

La propiedad numérica real de toda la población (por ejemplo, la media de la longitud del pétalo considerando todas las flores) se llama un parámetro. La estimación obtenida a partir de una muestra (como la media de esa propiedad en una muestra aleatoria de 30 flores) se llama un estadístico.

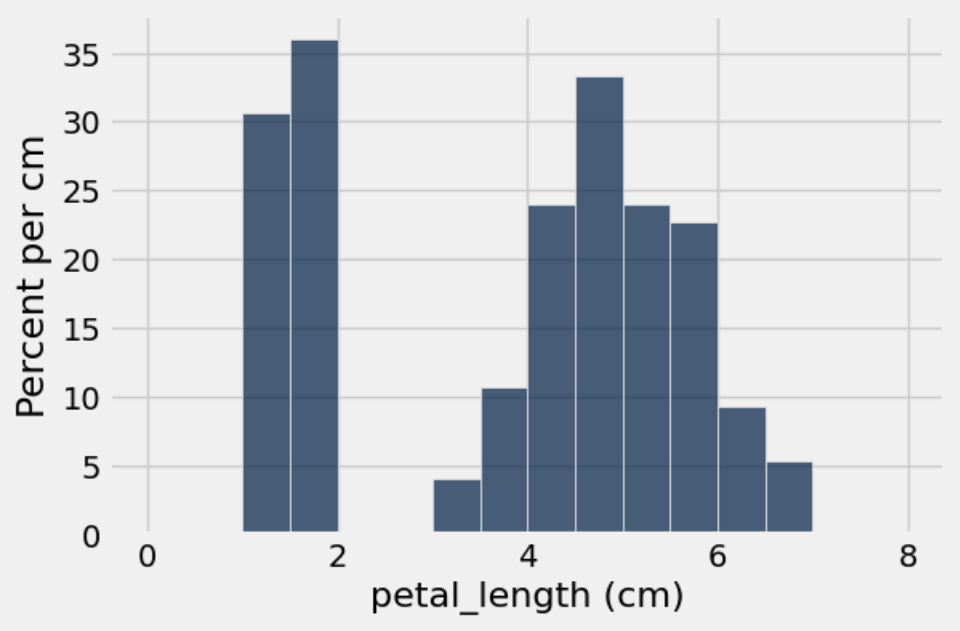
La utilidad de estos estadísticos dependerá de cómo se seleccionó la muestra. ¿Fue aleatoria? ¿Representa bien la diversidad de especies en el conjunto completo?

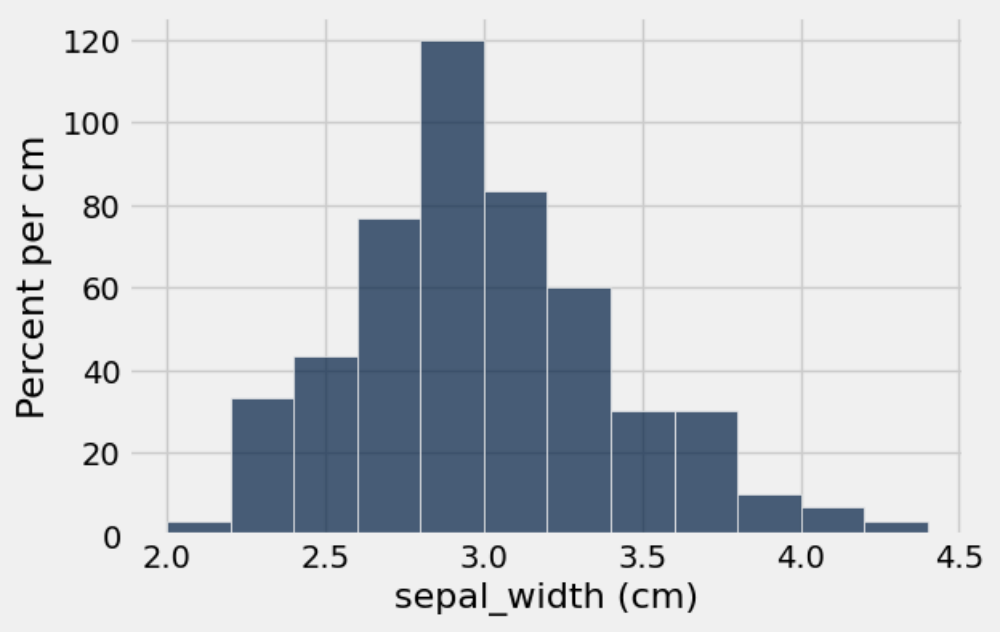
Hemos preparado algunos ejemplos de subconjuntos de muestra para que observes cómo se comparan con los valores reales del conjunto completo de datos Iris. Más adelante, te pediremos que generes tus propias muestras y analices cómo varían los estadísticos obtenidos.

Para ahorrar escritura y aumentar la claridad de tu código, empaquetaremos el código de análisis en unas pocas funciones. Esto será útil en el resto del laboratorio, ya que necesitaremos repetidamente crear histogramas y recopilar estadísticas resumidas de esos datos.

In [118]:

```
def histogramas_iris(tabla):
    # Definir los bins (intervalos) para cada variable
    bins_longitud_petalo = np.arange(0, 8.1, 0.5)       # Longitudes de pétalo entre 0 y 8 cm
    bins_ancho_sepalo = np.arange(2.0, 4.6, 0.2)        # Anchos de sépalo entre 2 y 4.5 cm

    # Histograma para la longitud del pétalo
    tabla.hist("petal_length", bins=bins_longitud_petalo, unit="cm")

    # Histograma para el ancho del sépalo
    tabla.hist("sepal_width", bins=bins_ancho_sepalo, unit="cm")

# Ejecutar la función con los datos completos
histogramas_iris(datos_iris)
```





Pregunta 3.1.

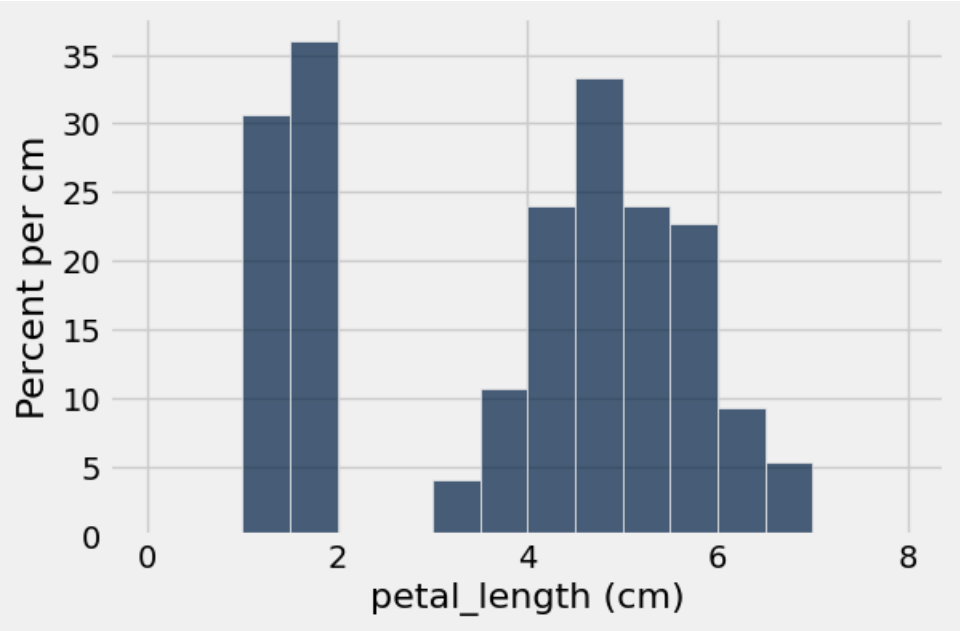
Crea una función llamada `calcular_estadisticos` que tome una tabla que contenga una columna "petal_length" (longitud del pétalo) y una columna "sepal_width" (ancho del sépalo), y que:

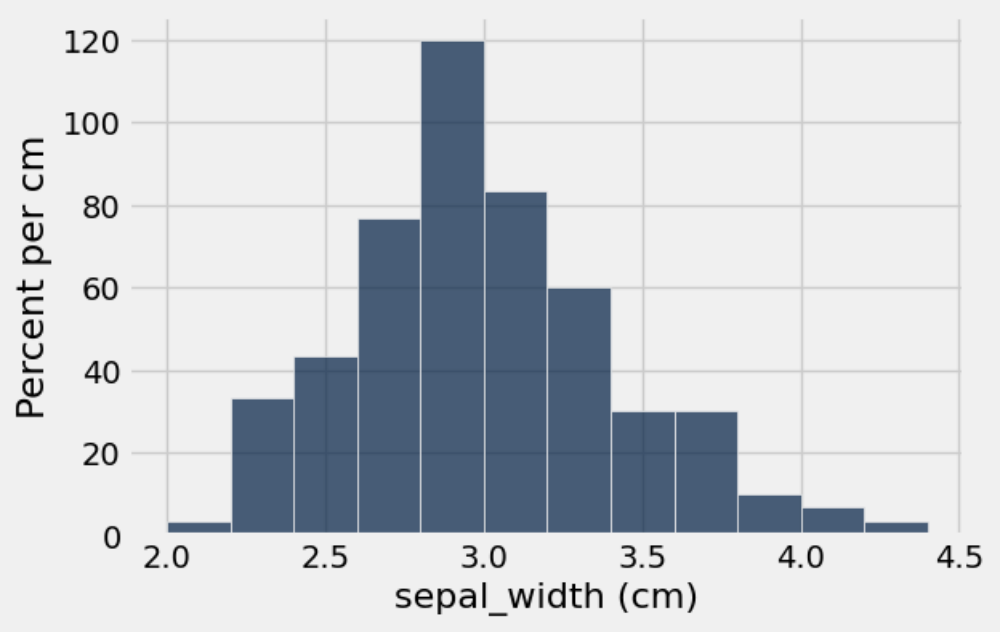
- Dibuje un histograma de la longitud del pétalo.
- Dibuje un histograma del ancho del sépalo.
- Devuelva un arreglo de dos elementos que contenga:
  - La longitud promedio del pétalo.
  - El ancho promedio del sépalo.

Puedes utilizar la función histogramas_iris para graficar los histogramas.

In [119]:

```
# BEGIN SOLUTION NO PROMPT

def calcular_estadisticos(tabla):
    # Dibujar histogramas
    histogramas_iris(tabla)

    # Calcular estadísticas
    longitud_promedio_petalo = np.mean(tabla.column("petal_length"))
    ancho_promedio_sepalo = np.mean(tabla.column("sepal_width"))

    # Retornar arreglo con los valores usando make_array
    return make_array(longitud_promedio_petalo, ancho_promedio_sepalo)
# END SOLUTION
""" # BEGIN PROMPT
def calcular_estadisticos(tabla):
    histogramas_iris(tabla)
    longitud_promedio_petalo = ...
    ancho_promedio_sepalo =...


    return ...
estadisticos = calcular_estadisticos(datos_iris)
""" # END PROMPT
estadisticos = calcular_estadisticos(datos_iris)
```





In [120]:

```
estadisticos
```

Out[120]:

```
array([ 3.75866667,  3.054     ])
```

In [ ]:

```
grader.check("q7")
```

Muestra aleatoria simple

Un enfoque consiste en tomar muestras de forma uniforme y aleatoria de los individuos en estudio. En una muestra aleatoria simple (SRS) sin reemplazo, nos aseguramos de que cada individuo tenga la misma probabilidad de ser seleccionado, y que ningún individuo sea elegido más de una vez. Imagina escribir el nombre de cada flor (o individuo) en una tarjeta, colocar todas las tarjetas en una caja, y mezclarla bien. Luego, sacas las tarjetas una por una sin devolverlas a la caja, hasta alcanzar el tamaño de muestra deseado.

Produciendo muestras aleatorias simples

A veces, es útil tomar muestras aleatorias incluso cuando tenemos los datos de toda la población. Esto nos ayuda a comprender la precisión del muestreo.

`sample`

El método de la tabla `sample` produce una muestra aleatoria de la tabla. Por defecto, selecciona al azar **con reemplazo** de las filas de una tabla. Muestrear con reemplazo significa que para cualquier fila seleccionada al azar, existe la posibilidad de que pueda ser seleccionada nuevamente si muestreamos múltiples veces. `Sample` toma como argumento el tamaño de la muestra y devuelve una **tabla** con solo las filas que fueron seleccionadas. Esto difiere de `np.random.choice`, que toma un arreglo y produce un valor aleatorio del mismo.

Ejecuta la celda a continuación para ver un ejemplo de llamada a `sample()` con un tamaño de muestra de 5, con reemplazo.

In [122]:

```
# Just run this cell
muestra_aleatoria = datos_iris.sample(5)
muestra_aleatoria.show()
```

sepal_length	sepal_width	petal_length	petal_width	species
6.8	2.8	4.8	1.4	versicolor
4.4	3	1.3	0.2	setosa
5.7	3	4.2	1.2	versicolor
6.8	3.2	5.9	2.3	virginica
5.9	3	5.1	1.8	virginica

El argumento opcional `with_replacement=False` se puede pasar a través de `sample()` para especificar que la muestra debe extraerse sin reemplazo.

Ejecuta la siguiente celda para ver un ejemplo de llamada a `sample()` con un tamaño de muestra de 5, sin reemplazo.

In [123]:

```
# Just run this cell

datos_iris.sample(5, with_replacement=False)
```

Out[123]:

sepal_length	sepal_width	petal_length	petal_width	species
5	2.3	3.3	1	versicolor
5	3.6	1.4	0.2	setosa
6.3	2.8	5.1	1.5	virginica
7	3.2	4.7	1.4	versicolor
6.3	2.3	4.4	1.3	versicolor

Pregunta 3.2 Genera una muestra aleatoria simple sin reemplazo de tamaño 44 a partir de la tabla datos_iris. Luego, realiza nuevamente tu análisis utilizando la función calcular_estadisticos que definiste anteriormente.

Ejecuta la celda varias veces para observar cómo cambian los histogramas y las estadísticas a medida que varía la muestra seleccionada.

- ¿Cuánto cambia la longitud promedio del pétalo entre muestras?
- ¿Y qué sucede con el ancho promedio del sépalo?

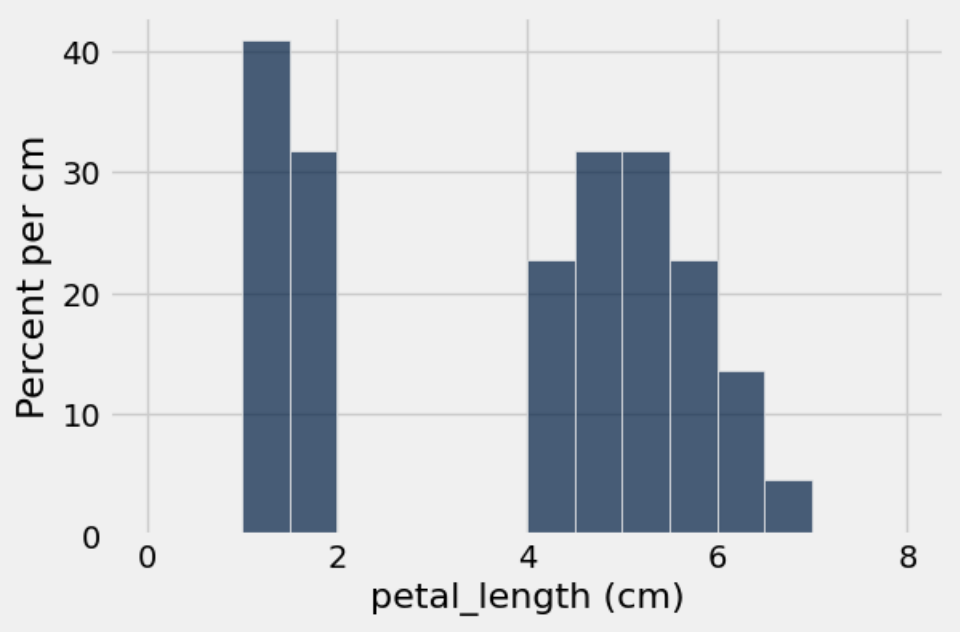
🅰️ Nota: "srs" significa muestra aleatoria simple (simple random sample) y "wor" significa sin reemplazo (without replacement).

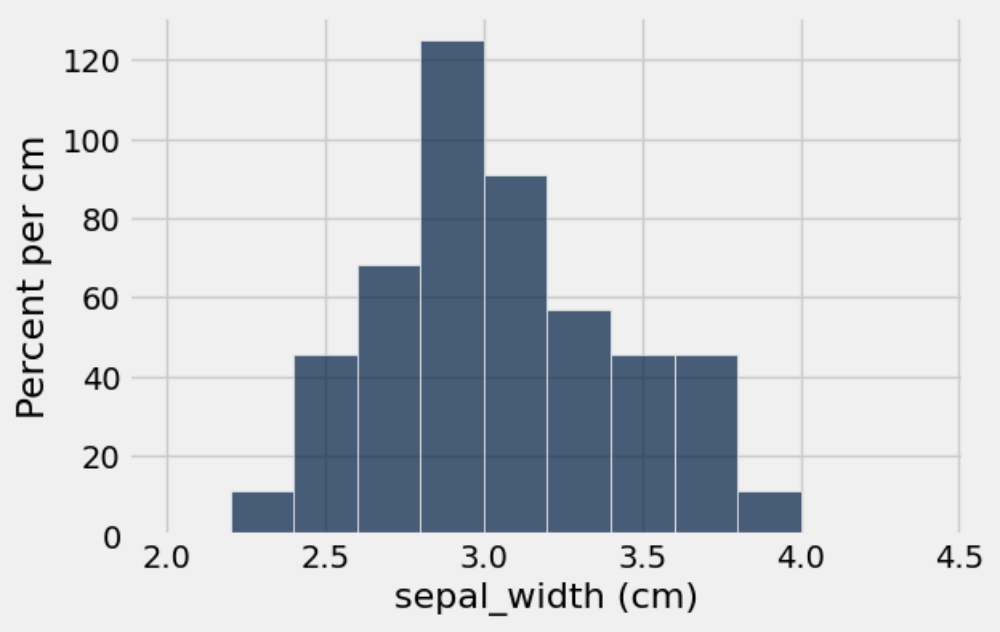
In [124]:

```
# BEGIN SOLUTION NO PROMPT
mi_muestra_aleatoria = datos_iris.sample(44, with_replacement=False)
# Calcular los estadísticos (longitud del pétalo y ancho del sépalo)
mis_estadisticos = calcular_estadisticos(mi_muestra_aleatoria)

# Mostrar el resultado
mis_estadisticos

# END SOLUTION
""" # BEGIN PROMPT
mi_muestra_aleatoria =...
# Calcular los estadísticos (longitud del pétalo y ancho del sépalo)
mis_estadisticos = ...
mis_estadisticos
""" # END PROMPT
```

Out[124]:

```
' # BEGIN PROMPT\nmi_muestra_aleatoria =...\n# Calcular los estadísticos (longitud del pétalo y ancho
del sépalo)\nmis_estadisticos = ...\nmis_estadisticos\n'
```





In [ ]:

```
grader.check("q8")
```

4. Más práctica con muestreo aleatorio

Más práctica para el muestreo aleatorio utilizando `np.random.choice`.

Simulaciones y Ciclos For (cont.)

Pregunta 4.1 Podemos utilizar `np.random.choice` para simular múltiples ensayos.

Ana decide jugar un juego lanzando un dado estándar de seis caras, donde su puntuación en cada lanzamiento está determinada por la cara que aparece. Ella quiere saber cuál sería su puntuación total si lanzara el dado 1000 veces. Escribe un código que simule su puntuación total después de 1000 lanzamientos.

Pista: Primero decide los valores posibles que puedes tomar en el experimento (valores de puntos en este caso). Luego utiliza `np.random.choice` para simular los lanzamientos de Ana. Finalmente, suma los lanzamientos para obtener la puntuación total de Ana.

In [127]:

```
# BEGIN SOLUTION NO PROMPT
caras_dado = np.array([1, 2, 3, 4, 5, 6])
# Simular los 1000 lanzamientos del dado
lanzamientos_ana = np.random.choice(caras_dado, size=1000)
# Calcular la puntuación total de Ana
puntuacion_total_ana = sum(lanzamientos_ana)
puntuacion_total_ana

# END SOLUTION
""" # BEGIN PROMPT
caras_dado=...
lanzamientos_ana=...
puntuacion_total_ana=...
puntuacion_total_ana
""" # END PROMPT
```

Out[127]:

```
' # BEGIN
PROMPT\ncaras_dado=...\nlanzamientos_ana=...\npuntuacion_total_ana=...\npuntuacion_total_ana\n'
```

In [ ]:

```
grader.check("q9")
```

Muestreo aleatorio simple (cont.)

El muestreo aleatorio simple es un método fundamental en estadística y análisis de datos donde cada elemento de la población tiene la misma probabilidad de ser seleccionado. Esto implica que no hay sesgo en la selección de muestras, asegurando que los resultados obtenidos sean representativos de la población completa. Este método es útil para extrapolar conclusiones a partir de una muestra sin introducir distorsiones por selección preferencial. Además, es una técnica clave empleada para garantizar la validez de los resultados en experimentos y estudios de investigación.

Imagen de un adorable gato llamado Winston

Sarita está muy feliz de que hayas terminado el laboratorio.

¡Has terminado el laboratorio!

Envío

Asegúrate de haber ejecutado todas las celdas en tu cuaderno en orden antes de ejecutar la celda de abajo, de modo que todas las imágenes/gráficas aparezcan en el resultado. La celda de abajo generará un archivo zip para que lo envíes. **¡Por favor, guarda antes de exportar!**

Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

These are some submission instructions.

In [ ]:

```
# Save your notebook first, then run this cell to export your submission.
grader.export(run_tests=True)
```